

The Essential

Concentrating on the essential

This week we shall discuss concepts that allow us to concentrate on the essential. In particular

- contexts
- contextual equivalence
- denotational semantics

Thank you to Matthew Alex, from the class in 2024/2025

Slide 1

Slide 2

Contexts

Motivation Consider the semantics from earlier

$$(W\text{-EXP.LEFT}) \quad \frac{\langle E_1, s \rangle \rightarrow_e \langle E'_1, s' \rangle}{\langle E_1 +_s E_2, s \rangle \rightarrow_e \langle E'_1 +_s E_2, s' \rangle}$$

$$(W\text{-EXP.RIGHT}) \quad \frac{\langle E_2, s \rangle \rightarrow_e \langle E'_2, s' \rangle}{\langle E_1 +_s E_2, s \rangle \rightarrow_e \langle E_1 +_s E'_2, s' \rangle}$$

$$(W\text{-EXP.VAR}) \quad \frac{}{\langle x, s \rangle \rightarrow_e \langle n, s \rangle} \quad s(x) = n$$

$$(W\text{-EXP.ADD}) \quad \frac{}{\langle n_1 +_s n_2, s \rangle \rightarrow_e \langle n_3, s \rangle} \quad n_3 = n_1 + n_2$$

What is the difference between rules like W-EXP.LEFT, W-EXP.RIGHT, and rules like W-EXP.VAR and W-EXP.ADD? Could we have written this semantics using fewer rules?

Slide 3

Expression Contexts

The set of **expression contexts**, $ExpCont$, is defined by

$$C^e \in ExpCont ::= \cdot \mid E +_s C^e \mid C^e +_s E \mid \dots$$

where $E \in Exp$.

The symbol $\cdot$ denotes the *context hole* of the expression context.

We write $C^e[\cdot]$ to emphasise the hole of C^e .

Exercise Give a definition of expression contexts that have zero, one, or many holes.

Some Simple Contexts

Slide 4

$$C_1^e[\cdot] \triangleq \cdot$$

$$C_2^e[\cdot] \triangleq \cdot +_s 2$$

$$C_3^e[\cdot] \triangleq (\cdot +_s 1) +_s 3$$

$$C_4^e[\cdot] \triangleq (3 +_s 4) +_s \cdot$$

Filling the Hole

Given an expression E and context $C^e[-]$, we can fill the hole with E yielding a new expression, written $C^e[E]$.

Slide 5

Context application, $C^e[E]$, for expression context C^e and simple expression E , is defined inductively on the structure of C^e by:

$$(\cdot)[E] \triangleq E$$

$$(C^e +_s E')[E] \triangleq C^e[E] +_s E'$$

$$(E' +_s C^e)[E] \triangleq E' +_s C^e[E]$$

Application Examples – 1**Slide 6**

$$C_1^e[3 +_s 4] = ??$$

$$C_2^e[3 +_s 4] = ??$$

$$C_3^e[3 +_s 4] = ??$$

$$C_4^e[3 +_s 4] = ??$$

Application Examples – 2**Slide 7**

$$C_1^e[3 +_s 4] = 3 +_s 4$$

$$C_2^e[3 +_s 4] = (3 +_s 4) +_s 2$$

$$C_3^e[3 +_s 4] = ((3 +_s 4) +_s 1) +_s 3$$

$$C_4^e[3 +_s 4] = (3 +_s 4) +_s (3 +_s 4)$$

Putting Expression Contexts to work

We revisit the semantics from earlier

$$(W\text{-EXP.EITHER}) \quad \frac{\langle E, s \rangle \rightarrow_e \langle E', s' \rangle}{\langle C^e[E], s \rangle \rightarrow_e \langle C^e[E'], s' \rangle}$$

Slide 8

$$(W\text{-EXP.VAR}) \quad \frac{}{\langle x, s \rangle \rightarrow_e \langle n, s \rangle} \quad s(x) = n$$

$$(W\text{-EXP.ADD}) \quad \frac{}{\langle n_1 +_s n_2, s \rangle \rightarrow_e \langle n_3, s \rangle} \quad n_3 = n_1 + n_2$$

Questions

1. We saved one rule at the expense of introducing expression contexts. Was it worth it?
2. Define contexts for booleans, and simplify the operational semantics accordingly.
3. Define contents for commands, and simplify the operational semantics accordingly.

What vs How

What vs How**Motivation**

What is the difference between C_1 , given below

$$i := i + 1$$
$$i := i + i$$

and C_2 , given below?

$$i := 2 * (i + 1)$$
Slide 9**Contextual Equivalence**

We now introduce an important idea in semantics: that of *contextual equivalence* between programs. Intuitively, we should be able to use equivalent programs (programs that behave in the same way) interchangeably: that is, if $P_1 \cong P_2$ (the symbol $\cong$ means that the programs are equivalent) and P_1 is used in some *context* (we write $C[P]$ for program P in program context $C[\cdot]$), then we should get the same effect if we replace P_1 with P_2 : that is, we expect $C[P_1] \cong C[P_2]$. To make this more precise, we say that a *context* $C^e[\cdot]$ is a program with a *hole* where you would ordinarily expect to see a subprogram.

Contextual Equivalence for Expressions

Definition We define **weak contextual equivalence** for expressions E_1 and E_2 :

$$E_1 \sim E_2 \triangleq \forall n. [E_1 \rightarrow^* n \Leftrightarrow E_2 \rightarrow^* n].$$

Definition We define **contextual equivalence** for expressions E_1 and E_2 :

$$E_1 \cong E_2 \triangleq \forall C^e[\cdot]. [C^e[E_1] \sim C^e[E_2]].$$

Lemma $\forall E_1, E_2. [E_1 \cong E_2 \Leftrightarrow E_1 \sim E_2]$

Questions.

1. Define contextual equivalence for booleans and for commands.
2. Does $C_1 \cong C_2$ hold?
3. Does $E_1 \cong E_2 \Leftrightarrow E_1 \sim E_2$ hold for all possible semantics?

Slide 10

Denotational Semantics

Denotational Semantics

Contextual equivalence allows us a succinct way to compare the outcome of execution of two expressions (or commands, or booleans). Nevertheless, it does not give describe *what* the expression (or command, or boolean) does.

Denotational Semantics describe the *meaning* of a program through a *mathematical object*. We start with **Denotational Semantics of *SimpleExp***, through a function

$$\llbracket _ \rrbracket : SimpleExp \rightarrow \mathbb{N}.$$

We also say that $\llbracket _ \rrbracket$ is the interpretation, and $\mathbb{N}$ is the *semantic domain* of simple expressions.

Question Does the structure of the semantic domain convey any information about execution of *SimpleExps*?

We will later discuss the denotational semantics of the While language.

Slide 11

Denotational Semantics for Simple Expressions

As we have seen, operational semantics talks about how an expression is evaluated to an answer. Denotational semantics, on the other hand, has grander aspirations. The denotational semantics of a language (such as *While* and *SimpleExp*) attempts to describe what a piece of program text ‘really means’. In the case of simple expressions, a piece of program text ‘is really’ a number, so we will define a function $\llbracket _ \rrbracket$ such that, for any expression E , $\llbracket E \rrbracket$ is a number, giving the meaning of E . Therefore, $\llbracket _ \rrbracket$ will be a function from expressions to numbers, and we write

$$\llbracket _ \rrbracket : SimpleExp \rightarrow \mathbb{N}$$

where $\mathbb{N}$ is a set of natural numbers. Given this function, the set $\mathbb{N}$ is called the *semantic domain* of *SimpleExp*, which just means it is the place where the meanings live. As we come to study more complex languages, we will find that we need more complex semantic domains. The construction and study of such domains is the subject of *domain theory*, an elegant mathematical theory which provides a foundation for denotational semantics; unfortunately, domain theory is beyond the scope of this course.

For now, notice that our choice of semantic domain has certain consequences for the semantics of our language: it implies that every expression will ‘mean’ exactly one number, so without even seeing the definition of $\llbracket _ \rrbracket$, someone looking at our semantics already knows that the language is (expected to be) normalizing and deterministic.

Denotational Semantics of Simple Expressions

We define $\llbracket _ \rrbracket : \text{SimpleExp} \rightarrow \mathbb{N}$ by induction on the structure of expressions:

$$\llbracket n \rrbracket = n$$

$$\llbracket E_1 +_s E_2 \rrbracket = \llbracket E_1 \rrbracket + \llbracket E_2 \rrbracket$$

The semantics is **compositional** in that the meaning of the compound expression $E_1 +_s E_2$ is given in terms of the meaning of its subexpressions, E_1 and E_2 .

Slide 12

Remarks

- The semantic domain is entirely separate from the syntax: for example, the set of natural numbers is a mathematical entity in its own right.
- The meaning of the compound term like $E_1 +_s E_2$ is given in terms of the meanings of its subterms. Hence, we have really given a meaning to the syntactic operation $+$. In this case, the meaning of $+$ is the usual addition function. We call a semantics *compositional* when it has this property, which lets us calculate meanings bit by bit, starting from the numerals and working up.

Calculating the Denotation**Slide 13**

$$\begin{aligned} \llbracket 1 +_s (2 +_s 3) \rrbracket &= \llbracket 1 \rrbracket + \llbracket (2 +_s 3) \rrbracket \\ &= 1 + \llbracket (2 +_s 3) \rrbracket \\ &= 1 + (\llbracket 2 \rrbracket + \llbracket 3 \rrbracket) \\ &= 1 + (2 + 3) \\ &= 6. \end{aligned}$$

The denotational semantics for expressions is particularly easy to work with, and much less cumbersome than the operational semantics. For example, it is easy to prove simple facts such as the associativity of the syntactic addition – as done in the next slide:

Associativity of $+_s$

Theorem For all E_1, E_2 and E_3 ,

$$\llbracket E_1 +_s (E_2 +_s E_3) \rrbracket = \llbracket (E_1 +_s E_2) +_s E_3 \rrbracket$$

Proof

Take any arbitrary expressions E_1, E_2, E_3 .

$$\begin{aligned} \llbracket E_1 +_s (E_2 +_s E_3) \rrbracket &= \llbracket E_1 \rrbracket + \llbracket E_2 +_s E_3 \rrbracket && \text{by def } \llbracket \dots \rrbracket \\ &= \llbracket E_1 \rrbracket + (\llbracket E_2 \rrbracket + \llbracket E_3 \rrbracket) && \text{by def } \llbracket \dots \rrbracket \\ &= (\llbracket E_1 \rrbracket + \llbracket E_2 \rrbracket) + \llbracket E_3 \rrbracket && \text{by associativity of } + \\ &= \llbracket E_1 +_s E_2 \rrbracket + \llbracket E_3 \rrbracket && \text{by def } \llbracket \dots \rrbracket \\ &= \llbracket (E_1 +_s E_2) +_s E_3 \rrbracket && \text{by def } \llbracket \dots \rrbracket \end{aligned}$$

Exercise State and prove a similar fact using the big-step semantics.

Slide 14

Contextual Equivalence

Recall that the denotational semantics is *compositional*: that is, the meaning of a large expression is built out of the meanings of its subphrases. It follows that each context determines a ‘function between meanings’: that is, for each $C[-]$, there is a function $f : \mathbb{N} \rightarrow \mathbb{N}$ such that

$$\llbracket C[E] \rrbracket = f(\llbracket E \rrbracket)$$

for any expression E .

A very important consequence of this is that

$$\forall C^e. [\llbracket E_1 \rrbracket = \llbracket E_2 \rrbracket \implies \llbracket C[E_1] \rrbracket = \llbracket C[E_2] \rrbracket]$$

Therefore, if we show

$$\llbracket E \rrbracket = n \text{ if and only if } E \Downarrow n$$

we can use our semantics to reason about contextual equivalence: that is, we will know that denotationally equivalent phrases are in fact contextually equivalent. For *SimpleExp*, this is indeed the case.

Denotational, Large Steps, Contextual Equivalence

Theorems

Slide 15

- $\forall E \in SimpleExp. [\llbracket E \rrbracket = n \iff E \Downarrow n]$.
- $\forall E_1, E_2 \in SimpleExp. [E_1 \cong E_2 \iff \llbracket E_1 \rrbracket = \llbracket E_2 \rrbracket]$.
- $\forall E_1, E_2 \in SimpleExp. \forall C^e[\cdot] \in ExpContext.$

$$[\llbracket E_1 \rrbracket = \llbracket E_2 \rrbracket \implies \llbracket C^e[E_1] \rrbracket = C^e[\llbracket E_2 \rrbracket]]$$

Denotational Semantics of *While*

We shall now begin to explore a denotational semantics of our simple language *While*. The first step is to choose our semantic domains.

Semantic Domains for *While*

We will define the **denotational semantics** of commands, expressions and booleans using the functions of the form

$$\llbracket - \rrbracket^c : \text{Com} \rightarrow ?$$

$$\llbracket - \rrbracket^e : \text{Exp} \rightarrow ?$$

$$\llbracket - \rrbracket^b : \text{Bool} \rightarrow ?$$

We need to choose our semantic domains.

Slide 16

Semantic Domain for Commands

Let us first focus on commands, which provide the biggest difference between *While* and *SimpleExp*. Let Σ be the set of all states. Define $\Sigma_{\perp}$ to be the set $\Sigma \cup \{\perp\}$: that is, the set Σ together with the extra element $\perp$, called *undefined* or *bottom*, which represents the stuck computation or an infinite loop.

Semantic Domain for *While* Commands

Let Σ be the set of all states, and define $\Sigma_{\perp}$ as the set $\Sigma \cup \{\perp\}$

The **semantic domain of commands** is given by the set of **state transformers** defined by

$$\mathcal{ST} \triangleq [\Sigma \rightarrow \Sigma_{\perp}] :$$

that is, the set of (total) functions which take a starting state and return either a final state, or the element $\perp$ (called **undefined** or **bottom**), to indicate that the computation got stuck or looped for ever.

Slide 17

A Note on Notation The metavariable s , and variants of it like s' and so on, will be used to range over *proper states*, not including $\perp$. So, if we say that $f(s) = s'$, it is implicit that $s' \neq \perp$.

Other Semantic Domains

Semantic Domains for Expressions and Booleans

The *semantic domain of expressions* is

$$\mathcal{SE} \triangleq [\Sigma \rightarrow \mathbb{N}_\perp]$$

The *semantic domain of booleans* is

$$\mathcal{SB} \triangleq [\Sigma \rightarrow \mathbb{B}_\perp]$$

where $\mathbb{B} = \{\text{true}, \text{false}\}$.

Slide 18

Semantic functions

We shall now explore three semantic functions, one for each category in the grammar for *While*.

Semantic Functions for *While*

$$\llbracket - \rrbracket^c : \text{Com} \rightarrow \mathcal{ST}$$

$$\llbracket - \rrbracket^e : \text{Exp} \rightarrow \mathcal{SE}$$

$$\llbracket - \rrbracket^b : \text{Bool} \rightarrow \mathcal{SB}$$

Slide 19

Remember that

$$\mathcal{ST} \triangleq [\Sigma \rightarrow \Sigma_{\perp}] \quad \mathcal{SE} \triangleq [\Sigma \rightarrow \mathbb{N}_{\perp}] \quad \mathcal{SB} \triangleq [\Sigma \rightarrow \mathbb{B}_{\perp}]$$

Notation: We use st, st' etc for state transformers, se, se' etc for expression denotations, and sb, sb' etc for boolean denotations.

Discussion: What extra information do we obtain from the signatures of $\llbracket - \rrbracket^c$, $\llbracket - \rrbracket^e$, and $\llbracket - \rrbracket^b$? Does the signature of the large steps relation $\Downarrow_c, \Downarrow_e$, or $\Downarrow_b$ convey as much information? Similar question for $\rightarrow_c, \rightarrow_e$, or $\rightarrow_b$.

For expressions and booleans, note that our language allows an expression or a boolean to *depend upon* the store, but *not* to change it. Also, though expressions and booleans cannot get into infinite loops, they may become stuck, so we have to account for this in our choice of semantic domain.

Remark As before, fixing the semantic domains tells us something about the language. For example, using $\mathcal{ST}$ for the commands acknowledges the possibility of non-termination, but makes clear that the command will yield at most one final state in any given starting state. Similarly, our choice of domain for the booleans automatically eliminates any possibility of side-effects being caused by boolean expressions.

Denotational Semantics of Expressions The denotational semantics of expressions is defined by induction on the structure of expressions. We shall only give the variable case; the others follow as you would expect from adapting the denotational *SimpleExp*. The case for a variable is simple. The store is examined to see if there is a value for the variable: if so, this value is returned; if not, the expression is stuck and we return $\perp$.

Denotational Semantics of Expressions

$\llbracket - \rrbracket^e : \text{Exp} \rightarrow \mathcal{SE}$ is defined compositionally on the structure of Exp

$$\begin{aligned}\llbracket n \rrbracket^e(s) &\triangleq n \\ \llbracket x \rrbracket^e(s) &\triangleq \text{selup}(x)(s) \\ \llbracket E_1 +_s E_2 \rrbracket^e(s) &\triangleq \text{seplus}(\llbracket E_1 \rrbracket^e, \llbracket E_2 \rrbracket^e)(s)\end{aligned}$$

where

$$\begin{aligned}\text{selup} &: ID \rightarrow \mathcal{SE} \\ \text{seplus} &: \mathcal{SE} \times \mathcal{SE} \rightarrow \mathcal{SE}\end{aligned}$$

and where

$$\begin{aligned}\text{selup}(x)(s) &\triangleq \begin{array}{ll} s(x) & \text{if } s(x) \text{ is defined,} \\ \perp & \text{otherwise} \end{array} \\ \text{seplus}(se_1, se_2)(s) &\triangleq \begin{array}{ll} se_1(s) + se_2(s) & \text{if both } se_1(s) \text{ and } se_2(s) \text{ are defined,} \\ \perp & \text{otherwise} \end{array}\end{aligned}$$

Denotational Semantics of Booleans The function $\llbracket - \rrbracket^b : \text{Bool} \rightarrow \mathcal{SB}$ is defined inductively by induction on the structure of Bool . This is straightforward and left as an exercise.

Denotational Semantics of Commands The function $\llbracket - \rrbracket^c : \text{Cmd} \rightarrow \mathcal{ST}$ is defined inductively by induction on the structure of Cmd . In each case, we ask ourselves how the command transforms the state, and attempt to write down a function which captures our intuition. The next few slides give the definitions.

Denotational Semantics of Commands

$\llbracket - \rrbracket^c : \text{Com} \rightarrow \mathcal{ST}$ is defined compositionally:

Slide 21

$$\begin{aligned}
 \llbracket x := E \rrbracket^c(s) &= st_{asg}(\llbracket E \rrbracket^e, x)(s) \\
 \llbracket \text{skip} \rrbracket^c(c) &= st_{skp}(s) \\
 \llbracket C_1; C_2 \rrbracket^c(s) &= st_{seq}(\llbracket C_1 \rrbracket^c, \llbracket C_2 \rrbracket^c)(s) \\
 \llbracket \text{if } B \text{ then } C_1 \text{ else } C_2 \rrbracket^c(s) &= st_{cnd}(\llbracket B \rrbracket^b, \llbracket C_1 \rrbracket^c, \llbracket C_2 \rrbracket^c)(s) \\
 \llbracket \text{while } B \text{ do } C \rrbracket^c &= st_{while}(\llbracket B \rrbracket^b, \llbracket C \rrbracket^c)(s).
 \end{aligned}$$

We now need to define st_{asg} , st_{skp} , st_{seq} , st_{cnd} , and st_{while} .

The signature of st_{asg} , st_{skp} , st_{seq} , st_{cnd} , and st_{while}

$$\begin{array}{ll}
 \llbracket x := E \rrbracket^c(s) &= st_{asg}(\llbracket E \rrbracket^e, x)(s) \\
 \llbracket \text{skip} \rrbracket^c(c) &= st_{skp}(s) \\
 \text{Remember ... } \llbracket C_1; C_2 \rrbracket^c(s) &= st_{seq}(\llbracket C_1 \rrbracket^c, \llbracket C_2 \rrbracket^c)(s) \\
 \llbracket \text{if } B \text{ then } C_1 \text{ else } C_2 \rrbracket^c(s) &= st_{cnd}(\llbracket B \rrbracket^b, \llbracket C_1 \rrbracket^c, \llbracket C_2 \rrbracket^c)(s) \\
 \llbracket \text{while } B \text{ do } C \rrbracket^c &= st_{while}(\llbracket B \rrbracket^b, \llbracket C \rrbracket^c)(s).
 \end{array}$$

Slide 22

$$\begin{aligned}
 st_{asg} &: \dots \\
 st_{skp} &: \dots \\
 st_{seq} &: \dots \\
 st_{cnd} &: \dots \\
 st_{while} &: \dots
 \end{aligned}$$

The signature of st_{asg} , st_{skp} , st_{seq} , st_{cnd} , and st_{while}

	$\llbracket x := E \rrbracket^c(s)$	$= st_{asg}(\llbracket E \rrbracket^c, x)(s)$
	$\llbracket \text{skip} \rrbracket^c(c)$	$= st_{skp}(s)$
Remember ...	$\llbracket C_1; C_2 \rrbracket^c(s)$	$= st_{seq}(\llbracket C_1 \rrbracket^c, \llbracket C_2 \rrbracket^c)(s)$
	$\llbracket \text{if } B \text{ then } C_1 \text{ else } C_2 \rrbracket^c(s)$	$= st_{cnd}(\llbracket B \rrbracket^b, \llbracket C_1 \rrbracket^c, \llbracket C_2 \rrbracket^c)(s)$
	$\llbracket \text{while } B \text{ do } C \rrbracket^c$	$= st_{while}(\llbracket B \rrbracket^b, \llbracket C \rrbracket^c)(s).$

Slide 23

$$\begin{aligned}
 st_{asg} &: \mathcal{SE} \times Id \rightarrow \mathcal{ST} \\
 st_{skp} &: \mathcal{ST} \\
 st_{seq} &: \mathcal{ST} \times \mathcal{ST} \rightarrow \mathcal{ST} \\
 st_{cnd} &: \mathcal{SB} \times \mathcal{ST} \times \mathcal{ST} \rightarrow \mathcal{ST} \\
 st_{while} &: \mathcal{SB} \times \mathcal{ST} \rightarrow \mathcal{ST}
 \end{aligned}$$

Defining st_{asg} , st_{skp} , st_{seq} , st_{cnd} , and st_{while}

Take $se : \mathcal{SE}$, $sb : \mathcal{SB}$, $st, st_1, st_2 : \mathcal{ST}$, and $s : \Sigma$. Then

$$\begin{aligned}
 st_{asg}(se, x)(s) &\triangleq \text{if } se(s) \neq \perp \text{ then } s[x \mapsto se(s)] \text{ else } \perp \\
 st_{skp}(s) &\triangleq s \\
 st_{seq}(st_1, st_2)(s) &\triangleq \text{if } st_1(s) \perp \text{ then } \perp \text{ else } st_2(st_1(s)) \\
 st_{cnd}(sb, st_1, st_2)(s) &\triangleq \text{if } sb(s) = \perp \text{ then } \perp \text{ else} \\
 &\quad \text{if } sb(s) \text{ then } st_1(s) \text{ else } st_2(s) \\
 st_{cnd}(sb, st)(s) &\triangleq \text{if } sb(s) = \perp \text{ then } \perp \text{ else} \\
 &\quad \text{if } sb(s) \text{ then } \dots \text{ else } s
 \end{aligned}$$

Slide 24

State transformer for $x := 0; x := x +_s 1$

Slide 25

$$\llbracket x := 0; x := x +_s 1 \rrbracket^c(s) = ..$$

...

??

We calculate the meaning of the command $C_o = x := 0; x := x +_s 1$.

$$\begin{aligned} \llbracket C_o \rrbracket^c &= st_{seq}(\llbracket x := 0 \rrbracket^c, \llbracket x := x +_s 1 \rrbracket^c) && \text{def. of } \llbracket ..; ... \rrbracket^c \\ &= st_{seq}(st_{asg}(x, \llbracket x +_s 1 \rrbracket^e), st_{asg}(x, \llbracket 0 \rrbracket^e)) && \text{def. of } \llbracket .. := ... \rrbracket^c, \text{ twice} \end{aligned}$$

Now, for any state s , we have

$$\begin{aligned} \llbracket C_o \rrbracket^c(s) &= st_{seq}(st_{asg}(x, \llbracket x +_s 1 \rrbracket^e), st_{asg}(x, \llbracket 0 \rrbracket^e))(s) && \text{as per earlier calculation} \\ &= (st_{asg}(x, \llbracket x +_s 1 \rrbracket^e))(s[x \mapsto 0]) && \text{by def. of } st_{asg}, \text{ and } \llbracket 0 \rrbracket^e \\ &= s[x \mapsto 1] && \text{def } st_{asg}, \llbracket ... +_s ... \rrbracket^e, sel_{up} \end{aligned}$$

Compositionality

Slide 26

Denotational semantics is *compositional*

The meaning of a program is built up out of the meanings of its subprograms. Therefore, each of the command-forming operations in the language *While* has a denotational meaning. For example, the sequential operator ‘;’, which combines two commands into one, has as its meaning the function st_{seq} . It is reasonable to say that

$$\llbracket ; \rrbracket = st_{seq}.$$

Denotational semantics *gives meaning to the command-forming operations*, not just the commands.

Semantics of `while`

Now what about `while`? How can we write a ‘looping’ state transformer? Recall the trick that we used to give a small-step semantics to `while`:

$$\langle \text{while } B \text{ do } C, s \rangle \rightarrow \langle \text{if } B \text{ then } (C; \text{while } B \text{ do } C) \text{ else skip}, s \rangle.$$

This says that the way `(while B do C)` transforms the state is the same as the transformation given by

$$\text{if } B \text{ then } (C; \text{while } B \text{ do } C) \text{ else skip}$$

Can we use equation

$\llbracket \text{while } B \text{ do } C \rrbracket^c = \llbracket \text{if } B \text{ then } (C; \text{while } B \text{ do } C) \text{ else skip} \rrbracket^c$ as a definition? We are trying to define the semantics of program phrases by induction on their structure. That means, as usual, that when we define the semantics of a compound phrase, we may assume that the semantics of each of its subphrases have already been defined. Bearing this in mind, it is clear that each of the definitions we have given so far is well-defined: that is, the formula on the right-hand side denotes an element of the semantic domain.

The equation above is different. It contains, on the right, a reference to

$$\llbracket \text{while } B \text{ do } C \rrbracket^c$$

which we have *not* yet defined: it is not a subphrase of itself! So we have a circular definition, or to put it another way, we do not have a definition at all. However, it does help to think of a while statement as a conditional and analyse how $(\text{while } B \text{ do } C)$ transforms a state s .

Towards state transformer for while

Remember that

$$\llbracket \text{while } B \text{ do } C \rrbracket^c = \llbracket \text{if } B \text{ then } (C; \text{while } B \text{ do } C) \text{ else skip} \rrbracket^c.$$

However, this is *not* a definition – it is an equation

Nevertheless, this equation gives:

$$st_{while}(sb, st) = st_{cnd}(sb, st_{seq}(st, st_{while}(sb, st)), st_{skp})$$

For given sb and st , we define $W : \mathcal{ST} \rightarrow \mathcal{ST}$ as

$$W(st_w) \triangleq st_{cnd}(sb, st_{seq}(st, st_w), st_{skp})$$

This means that $st_{while}(sb, st)$ is a *fixpoint* of W , i.e.,

$$W(st_{while}(sb, st)) = st_{while}(sb, st).$$

Therefore, if we find a fixpoint of W , we will be done!

We shall give a quick overview of how domain theory allows us to construct fixpoints. More detailed discussions can be found in several textbooks (e.g., Glynn Winskill's) and in Andrew Pitts' notes online.

Slide 27

Domain Theory to the Rescue – simplified

Assume a set A , and a function $F : (A \rightarrow A_{\perp}) \rightarrow (A \rightarrow A_{\perp})$ such that

$$(*) \quad \forall f \in A \rightarrow A_{\perp}. \forall a \in A. [f(a) \neq \perp \rightarrow F(f)(a) = f(a)].$$

We are looking for fixpoint(s) of F .

We define a sequence of functions $f_i : A \rightarrow A_{\perp}$ ($i \in \mathbb{N}$), through

$$(**) \quad f_0 \triangleq \perp \quad f_{i+1} \triangleq F(f_i).$$

Then, define function $f_{\infty} : A \rightarrow A_{\perp}$, through:

$$(***) \quad f_{\infty}(a) \triangleq a', \quad \text{if for some } i \in \mathbb{N}. f_i(a) = a' \neq \perp, \\ \perp, \quad \text{otherwise.}$$

Fixpoint Theorem f_{∞} is well-defined and is a fixpoint of F . Moreover, for any g , further fixpoint of F , we have that $\forall a \in A. [f_{\infty}(a) \neq \perp \Rightarrow f_{\infty}(a) = g(a)]$

We can also read (**) as $f_0(a) \triangleq \perp \quad f_{i+1}(a) \triangleq (F(f_i))(a)$ for all $a \in A$.

We can prove that $\forall i, j \in \mathbb{N}. [i \leq j \wedge f_i(a) \neq \perp \rightarrow f_j(a) = f_i(a)]$. That is, the sequence f_i is a series of *approximations*.

Slide 28

Applying the Fixpoint Theorem

The function $W : \mathcal{ST} \rightarrow \mathcal{ST}$ from earlier, defined as

$$W(st_w) \triangleq st_{cnd}(sb, st_{seq}(st, st_w), st_{skip})$$

has the property that

$$(*) \quad \forall st_w \in \mathcal{ST}. \forall s \in State. [st_w(s) \neq \perp \rightarrow W(st_w)(s) = st_w(s)].$$

Slide 29

We define a sequence of functions $w_i : \mathcal{ST}$ ($i \in \mathbb{N}$), through

$$(**) \quad w_0 \triangleq \perp \quad w_{i+1} \triangleq W(w_i).$$

Then, define function $w_\infty : \mathcal{ST}$, through:

$$(***) \quad w_\infty(s) \triangleq s', \quad \text{if for some } i \in \mathbb{N}. w_i(s) = s' \neq \perp, \\ \perp, \quad \text{otherwise.}$$

By **Fixpoint Theorem**, the function w_∞ is the smallest fixpoint of W .

Therefore, $st_{while}(sb, st) = w_\infty$.

Constructing the Approximations

Take a boolean expression B , and a command C . Take W , and w_i, w_∞ defined as earlier.

Assume a special command, `crash`, with $\forall s \in \Sigma. [\llbracket \text{crash} \rrbracket^c(s) = \perp]$.

Define a sequence of commands, $C_i \in Comm$ for $i \in \mathbb{N}$ as follows

$$(**) \quad C_0 \triangleq \text{crash} \\ C_{i+1} \triangleq \text{if } B \text{ then } (C; C_i) \text{ else skip.}$$

Slide 30

Theorem $(***) \quad \forall i \in \mathbb{N}. [w_i = \llbracket C_i \rrbracket^c]$

Theorem

$$st_{while}(\llbracket B \rrbracket^b, \llbracket C \rrbracket^c)(s) = \llbracket C_i \rrbracket^c(s) \quad \text{if } \exists i \in \mathbb{N}. \llbracket C_i \rrbracket^c \neq \perp, \\ \perp \quad \text{otherwise.}$$

Example Approximate $\llbracket \text{while } k > 1 \text{ do } k := k - 1; m := m + m \rrbracket^c$ through a sequence of commands, C_0, C_1, C_2 and C_3 .

We use the shorthand $C_a \triangleq k := k - 1; m := m + m$.

Then we define

$C_0 \triangleq \text{crash}$

$C_1 \triangleq \text{if } k > 1 \text{ then } (C_a; \text{crash}) \text{ else skip}$

$C_2 \triangleq \text{if } k > 1 \text{ then } (C_a; \text{if } k > 1 \text{ then } (C_a; \text{crash}) \text{ else skip}) \text{ else skip}$

$C_3 \triangleq \text{if } k > 1 \text{ then } (C_a; (\text{if } k > 1 \text{ then } (C_a; \text{if } k > 1 \text{ then } (C_a; \text{crash}) \text{ else skip}) \text{ else skip})) \text{ else skip}$

...

Consider states $s_a \triangleq [k \mapsto 1, m \mapsto 7]$, $s_b \triangleq [k \mapsto 2, m \mapsto 7]$, and $s_c \triangleq [k \mapsto 3, m \mapsto 7]$.

Apply $\llbracket C_i \rrbracket^c$ to s_a , for $i \in \{0, 1, 2, 3\}$. Same for s_b , and for s_c .

$$\begin{array}{lll} \llbracket C_0 \rrbracket^c(s_a) = \perp & \llbracket C_0 \rrbracket^c(s_b) = \perp & \llbracket C_0 \rrbracket^c(s_c) = \perp \\ \llbracket C_1 \rrbracket^c(s_a) = s_a & \llbracket C_1 \rrbracket^c(s_b) = \perp & \llbracket C_1 \rrbracket^c(s_c) = \perp \\ \llbracket C_2 \rrbracket^c(s_a) = s_a & \llbracket C_2 \rrbracket^c(s_b) = [k \mapsto 1, m \mapsto 14] & \llbracket C_2 \rrbracket^c(s_c) = \perp \\ \llbracket C_3 \rrbracket^c(s_a) = s_a & \llbracket C_3 \rrbracket^c(s_b) = [k \mapsto 1, m \mapsto 14] & \llbracket C_3 \rrbracket^c(s_c) = [k \mapsto 1, m \mapsto 28] \end{array}$$

Exercise Prove that

- Does $\llbracket x := z; y := z \rrbracket^c = \llbracket y := z; x := z \rrbracket^c$ hold? Prove it if it does, and give a counterexample otherwise.
- $\llbracket C_1; (C_2; C_3) \rrbracket^c = \llbracket (C_1; C_2); C_3 \rrbracket^c$

Summary

- *Contexts* allow us to succinctly describe whole families of terms.
- *Contextual equivalence* compares the behaviour of pairs of terms in infinitely many contexts.
- *Denotational semantics*
 - meaning described through mathematical objects, rather than executions (as in operational), or properties guaranteed by execution (as in axiomatic),
 - the semantic domains convey information about determinism, termination, and other properties.
 - recursive entities require fixpoint construction.